Refactoring Classes to Functions

Who needs classes!

Refactor Like a Pro

1. **Understanding the Purpose:**
   - Get the vibe of class responsibilities
   - Spot methods for function transformation
2. **Decomposition:**
   - Break class into smaller, cohesive functions
   - Each function for a single task or purpose

3. **Data Handling:**
   - Convert class attributes to function parameters
   - Ensure clear data flow between functions
4. **State Management:**
   - Pass required state as function arguments
   - Reduce reliance on class attributes

5. **Static Methods and Class Methods:**
   - Transform into their standalone functions
   - Reduce reliance on class attributes
6. **Dependency Injection:**
   - Get explicit with function dependencies
   - Increase predictability and testability 🎸✨

7. **Testing:**
   - Pre and post-refactoring tests to keep the rhythm
   - Functions make unit testing smoother
8. **Naming Conventions:**
   - Function names following PEP 8
   - Reflective of their purpose

Composition to Functions

Example: Class using Composition

```
class Engine:
    def start(self):
        print("Engine started.")

class Car:
    def __init__(self):
        self.engine = Engine()

    def start_engine(self):
        self.engine.start()

my_car = Car()
my_car.start_engine()
```

Refactored to Functions:

```
def start_engine():
    print("Engine started.")

def start_car():
    start_engine()

start_car()
```

Aggregation to Functions

Example: Class using Aggregation

```
class Order:
    def __init__(self, items):
        self.items = items

    def calculate_total(self):
        total = 0
        for item in self.items:
            total += item.price
        return total

class Item:
    def __init__(self, name, price):
        self.name = name
        self.price = price

item1 = Item("Book", 10)
item2 = Item("Pen", 5)

order = Order([item1, item2])
total = order.calculate_total()
print(f"Total: ${total}")
```

Refactored to Functions:

```
def calculate_total(items):
    total = 0
    for item in items:
        total += item['price']
    return total

def create_item(name, price):
    return {'name': name, 'price': price}

item1 = create_item("Book", 10)
item2 = create_item("Pen", 5)

order_items = [item1, item2]
total = calculate_total(order_items)
print(f"Total: ${total}")
```

Inheritance to Functions

Example: Class using Inheritance

```
class Animal:
    def make_sound(self):
        pass

class Dog(Animal):
    def make_sound(self):
        print("Bark")

class Cat(Animal):
    def make_sound(self):
        print("Meow")

def animal_sound(animal):
    animal.make_sound()

dog = Dog()
cat = Cat()

animal_sound(dog)  # Output: Bark
animal_sound(cat)  # Output: Meow
```

Refactored to Functions:

```
def make_sound(sound):
    print(sound)

make_sound("Bark")  # Output: Bark
make_sound("Meow")  # Output: Meow
```